

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Обработка изображений в интеллектуальных системах»
Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнил:

Студент 4 курса

Группы ИИ-26

Хомиченко И.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Ход работы

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

В-т	Выборка	Оптимизатор	Предобученная архитектура
7	Fashion-MNIST	Adam	ResNet34

Код программы:

```
import argparse
import csv
import random
import time
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch
from PIL import Image, ImageOps
from torch import nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms
from torchvision.models import ResNet34_Weights, resnet34

BASE_DIR = Path(__file__).resolve().parent
```

```

SEED = 42
CLASSES = [
    "Футболка", "Брюки", "Свитер", "Платье", "Пальто",
    "Сандалии", "Рубашка", "Кроссовки", "Сумка", "Ботинки"
]
COLORS = ["#3675b5", "#e28b36"]
REFERENCES = {"WRN40-4": 96.7, "DenseNet-BC": 95.4}
REFERENCE_URL = "https://github.com/zalandoresearch/fashion-mnist#benchmark"

```

```

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Flatten(), nn.Linear(32 * 8 * 8, 128), nn.ReLU(),
            nn.Dropout(0.25), nn.Linear(128, 10)
        )

    def forward(self, x):
        return self.layers(x)

def build_resnet(pretrained=True):
    model = resnet34(weights=ResNet34_Weights.DEFAULT if pretrained else None)
    original = model.conv1
    model.conv1 = nn.Conv2d(
        1, original.out_channels, original.kernel_size,
        original.stride, original.padding, bias=False
    )
    with torch.no_grad():
        model.conv1.weight.copy_(original.weight.mean(dim=1, keepdim=True))
    model.fc = nn.Linear(model.fc.in_features, 10)
    return model

def make_transform(size):
    return transforms.Compose([
        transforms.Grayscale(num_output_channels=1),
        transforms.Resize((size, size)),
        transforms.ToTensor(),
        transforms.Normalize((0.2860,), (0.3530,))
    ])

def make_loaders(transform, train_indices, val_indices, batch_size, device):
    training = datasets.FashionMNIST(
        BASE_DIR / "data", train=True, download=True, transform=transform
    )

```

```

testing = datasets.FashionMNIST(
    BASE_DIR / "data", train=False, download=True, transform=transform
)
settings = dict(batch_size=batch_size, num_workers=0, pin_memory=device.type == "cuda")
train_loader = DataLoader(
    Subset(training, train_indices), shuffle=True,
    generator=torch.Generator().manual_seed(SEED), **settings
)
val_loader = DataLoader(Subset(training, val_indices), **settings)
test_loader = DataLoader(testing, **settings)
return train_loader, val_loader, test_loader

```

```

def run_epoch(model, loader, criterion, device, optimizer=None):
    model.train(optimizer is not None)
    total_loss = correct = total = 0
    with torch.set_grad_enabled(optimizer is not None):
        for inputs, labels in loader:
            inputs, labels = inputs.to(device), labels.to(device)
            if optimizer is not None:
                optimizer.zero_grad(set_to_none=True)
            logits = model(inputs)
            loss = criterion(logits, labels)
            if optimizer is not None:
                loss.backward()
                optimizer.step()
            total_loss += loss.item() * labels.size(0)
            correct += (logits.argmax(1) == labels).sum().item()
            total += labels.size(0)
    return total_loss / total, 100 * correct / total

```

```

def train_model(model, loaders, name, args, device, output_dir):
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=args.lr)
    history = []
    best_accuracy = -1
    checkpoint = output_dir / f"{name.lower()}_best.pth"
    started = time.perf_counter()
    print(f"\n{name} | Adam | эпох: {args.epochs} | lr: {args.lr}")
    print(f"{'Эпоха':>7} {'Ошибка train':>14} {'Ошибка val':>12} {'Train, %':>10} {'Val, %':>10} {'Минуты':>9}")
    for epoch in range(1, args.epochs + 1):
        train_loss, train_accuracy = run_epoch(model, loaders[0], criterion, device, optimizer)
        val_loss, val_accuracy = run_epoch(model, loaders[1], criterion, device)
        history.append([epoch, train_loss, val_loss, train_accuracy, val_accuracy])
        minutes = (time.perf_counter() - started) / 60
        print(f"{'epoch':>7} {'train_loss':>14.4f} {'val_loss':>12.4f} {'train_accuracy':>10.2f} {'val_accuracy':>10.2f} {'minutes':>9.1f}", flush=True)
        if val_accuracy > best_accuracy:

```

```

        best_accuracy = val_accuracy
        torch.save(model.state_dict(), checkpoint)
    model.load_state_dict(torch.load(checkpoint, map_location=device, weights_only=True))
    test_loss, test_accuracy = run_epoch(model, loaders[2], criterion, device)
    with (output_dir / f"{name.lower()}_history.csv").open("w", encoding="utf-8", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["epoch", "train_loss", "val_loss", "train_accuracy", "val_accuracy"])
        writer.writerows(history)
    return history, test_loss, test_accuracy

def plot_training(histories, results, output_dir):
    fig, axes = plt.subplots(2, 2, figsize=(13, 9))
    for (name, history), color in zip(histories.items(), COLORS):
        data = np.asarray(history)
        axes[0, 0].plot(data[:, 0], data[:, 1], color=color, label=f"{name}: обучение")
        axes[0, 0].plot(data[:, 0], data[:, 2], "--", color=color, label=f"{name}: валидация")
        axes[0, 1].plot(data[:, 0], data[:, 3], color=color, label=f"{name}: обучение")
        axes[0, 1].plot(data[:, 0], data[:, 4], "--", color=color, label=f"{name}: валидация")
    for ax, title, ylabel in zip(axes[0], ["Ошибка обучения", "Точность обучения"],
["CrossEntropyLoss", "Точность, %"]):
        ax.set(title=title, xlabel="Эпоха", ylabel=ylabel)
        ax.legend(fontsize=8)
        ax.grid(alpha=0.2)
    names = list(results)
    values = [results[name][1] for name in names]
    bars = axes[1, 0].barh(names, values, color=COLORS)
    axes[1, 0].set(title="Наши модели: тестовая выборка", xlabel="Точность, %", xlim=(0, 105))
    for bar, value in zip(bars, values):
        axes[1, 0].text(value + 0.5, bar.get_y() + bar.get_height() / 2, f"{value:.2f}%", va="center")
    axes[1, 1].axis("off")
    reference_text = "Опубликованные ориентиры Fashion-MNIST\n\n"
    reference_text += "\n".join(f"{name}: {accuracy:.2f}%" for name, accuracy in REFERENCES.items())
    reference_text += "\n\nИсточник: benchmark zalandoresearch/fashion-mnist\n\nПротоколы обучения\nотличаются.\nЭто ориентиры, а не подтверждённый текущий SOTA."
    axes[1, 1].text(0.03, 0.85, reference_text, va="top", fontsize=10)
    fig.suptitle("Лабораторная работа 2 · Вариант 7 · Fashion-MNIST", fontsize=15)
    fig.tight_layout()
    fig.savefig(output_dir / "training_dashboard.png", dpi=160)

def visualize_prediction(models, preprocessing, image, title, filename, device, output_dir):
    fig, axes = plt.subplots(1, 3, figsize=(15, 6), gridspec_kw={"width_ratios": [1, 1.5, 1.5]})
    axes[0].imshow(image, cmap="gray", vmin=0, vmax=255)
    axes[0].set_title(title)
    axes[0].axis("off")
    print(f"\n{title}")
    for ax, (name, model), color in zip(axes[1:], models.items(), COLORS):
        model.eval()
        tensor = preprocessing[name](image).unsqueeze(0).to(device)

```

```

with torch.inference_mode():
    probabilities = model(tensor).softmax(dim=1)[0].cpu().numpy()
    prediction = int(probabilities.argmax())
    ax.barh(CLASSES, probabilities * 100, color=color)
    ax.invert_yaxis()
    ax.set(xlim=(0, 100), xlabel="Вероятность softmax, %", title=f"{name}: {CLASSES[prediction]}")
    print(f" {name:<10} → {CLASSES[prediction]} ({probabilities[prediction] * 100:.2f}%)")
    for index in np.argsort(probabilities)[-3:][::-1]:
        print(f" {CLASSES[index]:<12}: {probabilities[index] * 100:6.2f}%")
fig.tight_layout()
fig.savefig(output_dir / filename, dpi=160)

```

```

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--epochs", type=int, default=5)
    parser.add_argument("--batch-size", type=int, default=32)
    parser.add_argument("--lr", type=float, default=0.0001)
    parser.add_argument("--resnet-size", type=int, default=96)
    parser.add_argument("--image", type=Path, default=Path("bag.png"))
    parser.add_argument("--invert-image", action="store_true")
    parser.add_argument("--no-show", action="store_true")
    args = parser.parse_args()
    if args.epochs < 1 or args.batch_size < 1 or args.lr <= 0 or args.resnet_size < 32:
        parser.error("Эпохи и batch-size должны быть положительными, lr > 0, resnet-size >= 32.")
    image_path = args.image
    if image_path is not None:
        if not image_path.is_absolute():
            image_path = BASE_DIR / image_path
        if not image_path.is_file():
            parser.error(f"Изображение не найдено: {image_path}")
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(SEED)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    output_dir = BASE_DIR / "results_variant7"
    output_dir.mkdir(exist_ok=True)
    indices = torch.randperm(60000, generator=torch.Generator().manual_seed(SEED)).tolist()
    train_indices, val_indices = indices[:54000], indices[54000:]
    preprocessing = {"SimpleCNN": make_transform(32), "ResNet34": make_transform(args.resnet_size)}
    print(f"БАРИАНТ 7 | Fashion-MNIST | Adam | ResNet34\nУстройство: {device}")
    print("Разбиение: обучение 54000 / валидация 6000 / тест 10000")
    print(f"Размер входа: SimpleCNN 32×32, ResNet34 {args.resnet_size}×{args.resnet_size}")
    histories, results, models = {}, {}, {}
    for name in preprocessing:
        loaders = make_loaders(preprocessing[name], train_indices, val_indices, args.batch_size, device)
        model = (SimpleCNN() if name == "SimpleCNN" else build_resnet()).to(device)
        history, loss, accuracy = train_model(model, loaders, name, args, device, output_dir)

```

```

histories[name], results[name] = history, (loss, accuracy)
models[name] = model.cpu()
report = ["Вариант 7: Fashion-MNIST / Adam / ResNet34", "Тест лучших по валидации моделей:"]
report += [f"{name}: ошибка {loss:.4f}; точность {accuracy:.2f}%" for name, (loss, accuracy) in
results.items()]
difference = results["ResNet34"][1] - results["SimpleCNN"][1]
report += [f"Разница ResNet34 – SimpleCNN: {difference:+.2f} п.п."]
report += [f"Опубликованный ориентир {name}: {value:.2f}%" for name, value in
REFERENCES.items()]
report += [f"Источник: {REFERENCE_URL}", "Опубликованные результаты получены с
другими протоколами; текущий SOTA не установлен."]
report += [f"Параметры: epochs={args.epochs}, batch_size={args.batch_size}, lr={args.lr},
resnet_size={args.resnet_size}, seed={SEED}"]
text = "\n".join(report)
print(f"\n{text}")
(output_dir / "results.txt").write_text(text, encoding="utf-8")
plot_training(histories, results, output_dir)
for model in models.values():
    model.to(device)
raw_test = datasets.FashionMNIST(BASE_DIR / "data", train=False, download=False)
image, label = raw_test[random.randrange(len(raw_test))]
visualize_prediction(models, preprocessing, image, f"Тестовый пример: {CLASSES[label]}",
"test_prediction.png", device, output_dir)
if image_path is not None:
    with Image.open(image_path) as source:
        image = ImageOps.exif_transpose(source).convert("L")
    if args.invert_image:
        image = ImageOps.invert(image)
    visualize_prediction(models, preprocessing, image, f"Внешнее изображение: {image_path.name}",
"custom_prediction.png", device, output_dir)
else:
    print("Для классификации своего фото запустите с параметром --image clothing.png")
print(f"\nГрафики, веса и результаты сохранены: {output_dir}")
if args.no_show:
    plt.close("all")
else:
    plt.show()
if __name__ == "__main__":
    main()

```

Результат программы:

ВАРИАНТ 7 | Fashion-MNIST | Adam | ResNet34

Устройство: cuda

Разбиение: обучение 54000 / валидация 6000 / тест 10000

Размер входа: SimpleCNN 32×32, ResNet34 96×96

SimpleCNN | Adam | эпох: 5 | lr: 0.0001

Эпоха	Ошибка train	Ошибка val	Train, %	Val, %	Минуты
1	0.6783	0.4511	76.29	84.12	0.2
2	0.4406	0.4015	84.19	85.67	0.5

3	0.3880	0.3654	86.26	86.95	0.8
4	0.3576	0.3365	87.16	87.75	1.1
5	0.3361	0.3162	87.91	88.45	1.4

ResNet34 | Adam | эпох: 5 | lr: 0.0001

Эпоха	Ошибка train	Ошибка val	Train, %	Val, %	Минуты
1	0.2839	0.2156	90.15	92.58	1.7
2	0.1756	0.1981	93.81	92.35	3.3
3	0.1347	0.1556	95.26	94.47	4.9
4	0.1041	0.1474	96.22	94.85	6.5
5	0.0770	0.1644	97.22	94.77	8.1

Вариант 7: Fashion-MNIST / Adam / ResNet34

Тест лучших по валидации моделей:

SimpleCNN: ошибка 0.3313; точность 87.91%

ResNet34: ошибка 0.1730; точность 94.12%

Разница ResNet34 – SimpleCNN: +6.21 п.п.

Опубликованный ориентир WRN40-4: 96.70%

Опубликованный ориентир DenseNet-BC: 95.40%

Источник: <https://github.com/zalandoresearch/fashion-mnist#benchmark>

Опубликованные результаты получены с другими протоколами; текущий SOTA не установлен.

Параметры: epochs=5, batch_size=32, lr=0.0001, resnet_size=96, seed=42

Тестовый пример: Сандалии

SimpleCNN → Сандалии (95.66%)

Сандалии : 95.66%

Ботинки : 4.33%

Кроссовки : 0.01%

ResNet34 → Сандалии (100.00%)

Сандалии : 100.00%

Кроссовки : 0.00%

Сумка : 0.00%

Внешнее изображение: bag.png

SimpleCNN → Сумка (68.42%)

Сумка : 68.42%

Рубашка : 16.37%

Футболка : 5.19%

ResNet34 → Сумка (99.94%)

Сумка : 99.94%

Рубашка : 0.03%

Свитер : 0.01%

Figure 1

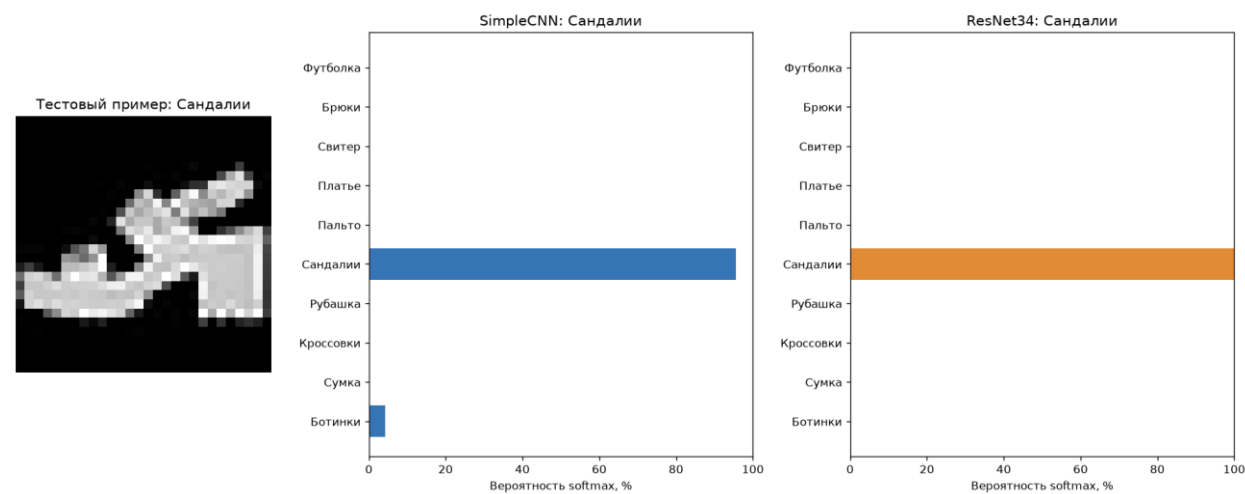
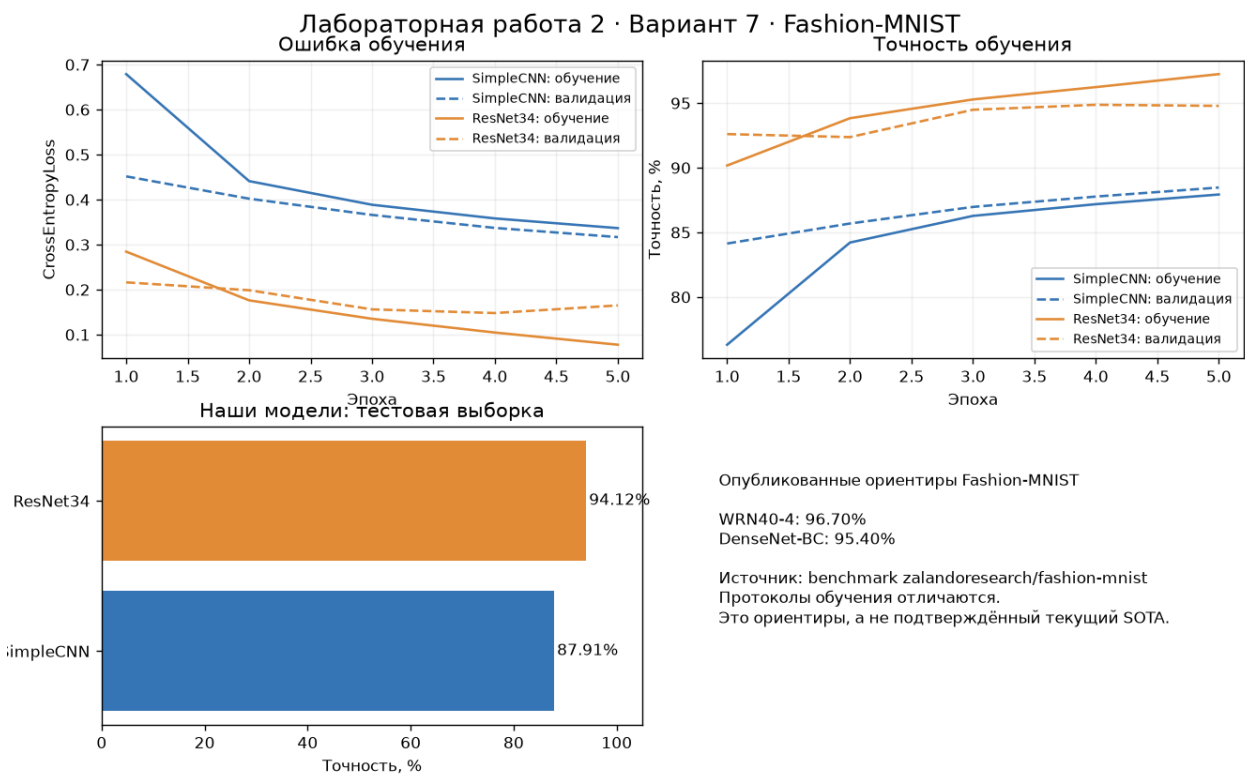
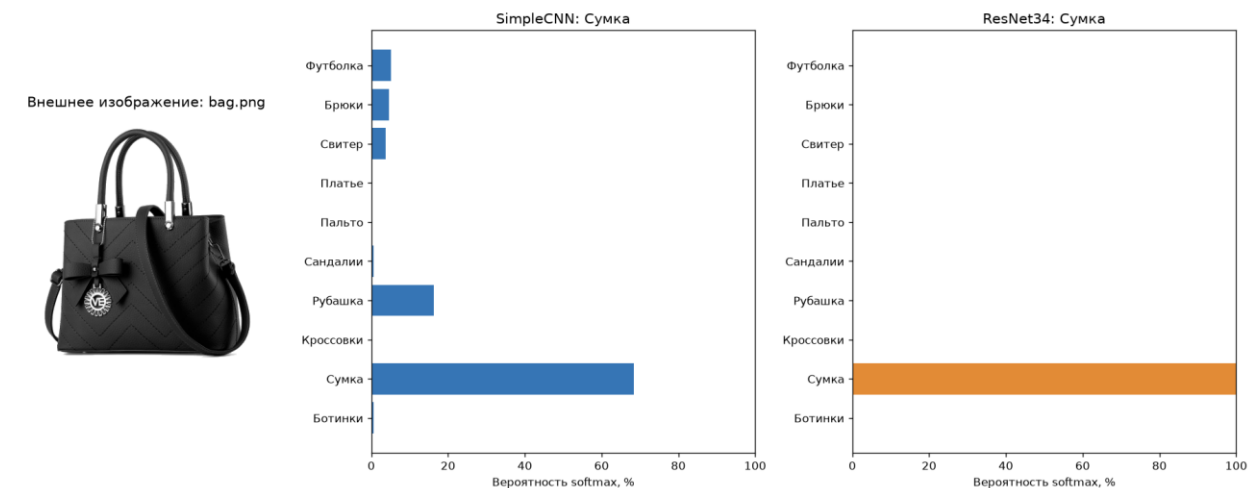


Figure 3



Сравнение с Лаб1:

ResNet34 превзошла сеть из первой лабораторной на 2,64 процентного пункта. Число ошибок сократилось с 852 до 588 — примерно на 31%.

Лучшие результаты валидации получены на четвёртой эпохе. На пятой ошибка валидации выросла при снижении ошибки обучения — это признак начинающегося переобучения. Для тестирования программа использовала веса четвёртой эпохи.

Для сравнения рассмотрены DenseNet-BC — 95,40%, WRN40-4 — 96,70% и заявленный авторами статьи результат CNN-3-128 с аугментацией — 99,65%. Условия экспериментов отличаются, поэтому эти значения приведены как опубликованные ориентиры.

Вывод: Осуществил обучение НС, сконструированных на базе предобученных архитектур НС.